EL APPLICATIONS WEB DISTRIBUÉES · MT-41

Arène

Plateforme de hackathons & bounties

REVUE DE CADRAGE ET D'ARCHITECTURE · SÉANCE 4, JALON 1

VALIDATION FINALE · 2026-2027

Ce que nous validons aujourd'hui

01

Cadrage

Idée, besoin, utilisateurs, scénario de bout en bout, périmètre MoSCoW.

02

Architecture commune

Diagramme global, six microservices, une base par service, communication synchrone et asynchrone, stack.

03

Les tâches, une par une

Chaque membre présente son service : rôle, données, communications et contrats d'API.

04

Pilotage

Gestion de projet, organisation Git, feuille de route et risques.

Ordre de passage : **M1** Challenges → **M2** Participants → **M3** Soumissions → **M4** Évaluation → **M5** Réputation → **Équipe** Vérification (MS5), infrastructure et Frontend.

L’idée et le besoin

Le problème

- Une entreprise a un **besoin logiciel ponctuel**.
- Elle ne veut ni **recruter une équipe**, ni payer des freelances au forfait.
- Les développeurs veulent des **projets réels**, un **prix** et de la **visibilité**.

La solution : Arène

- L’entreprise publie un **défi** : spec, **tests d’acceptation** et **prime (top 3)**.
- Les développeurs soumettent leur **dépôt GitHub**.
- Un **moteur de vérification** rejette automatiquement les échecs.
- Les **experts** évaluent et désignent les **gagnants**.

> Valeur métier : réduire le coût d’accès à du logiciel sur mesure, et offrir aux développeurs un portfolio *vérifiable*.

Les utilisateurs

Entreprise

Obtenir un logiciel conforme, à coût maîtrisé, avec une qualité garantie.

Développeur / Équipe

Projets réels, prime, visibilité et profil valorisant pour l'embauche.

Expert

Évaluer les projets sur des critères clairs et pondérés.

Admin

Modérer, gérer les litiges et les comptes.

Rôles Keycloak : DEVELOPER ENTERPRISE EXPERT ADMIN

Chaque route est protégée par rôle (`@PreAuthorize`) : l'entreprise publie et finance, le développeur soumet, l'expert évalue, l'administrateur modère.

Scénario de bout en bout

01

L'entreprise publie un défi : spec, tests, primes, date limite.

02

Le développeur s'inscrit, seul ou en équipe.

03

Il soumet son dépôt GitHub, avec un commit figé.

04

Le moteur exécute les tests et rend un verdict.

05

Les experts notent les projets retenus.

06

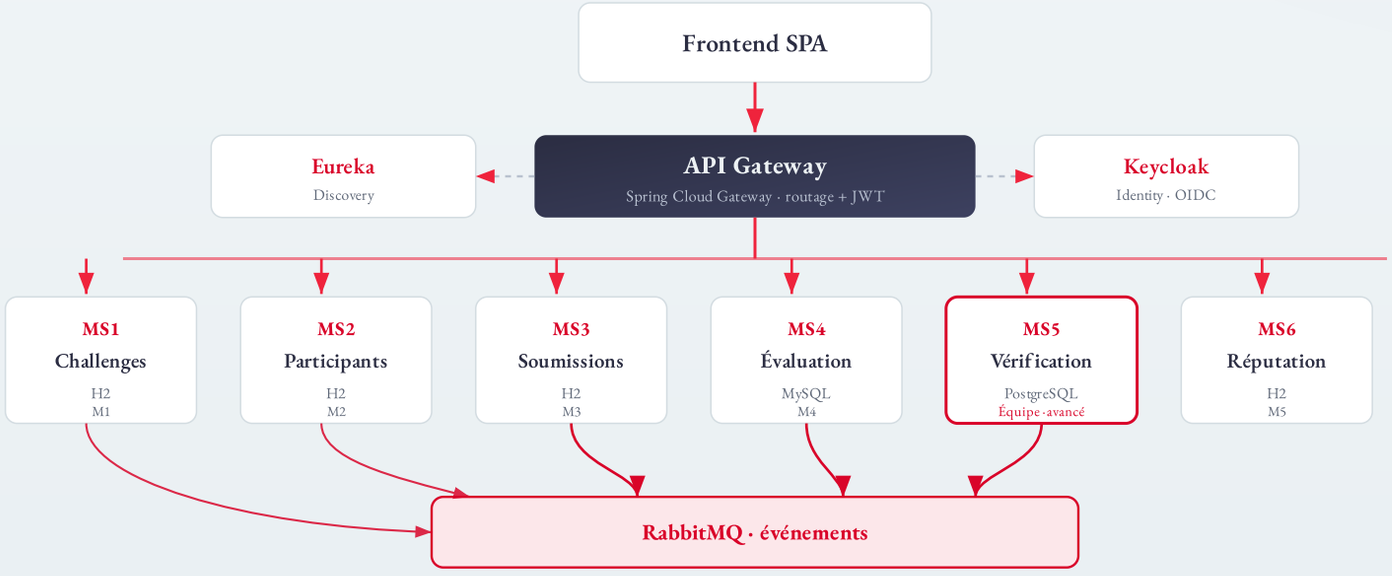
Les gagnants (top 3) sont désignés et récompensés.

> Un FAIL aux tests d'acceptation déclenche un **rejet automatique**. Les logs restent consultables par l'équipe.

Priorisation MoSCoW

Must	Publier un défi · s'inscrire · soumettre un dépôt · **exécuter les tests** · verdict PASS/FAIL et rejet automatique · notation · classement · gagnants · auth Keycloak · Gateway et Eureka.
Should	Réputation et badges · feedback du jury · tableau de bord entreprise · notifications d'événements · anti-plagiat.
Could	Appariement experts et défis · résumé IA des soumissions · rédaction assistée des tests · messagerie · statistiques.
Won't	Paiement réel (simulé) · dépôts GitHub privés · exécution multi-langages · application mobile.

Architecture globale



8 événements · producteurs MS1 à MS5 · MS6 consommateur

cœur : submission.submitted → verification.completed → winners.declared

REST (via Gateway)

Événements (asynchrone)

Liaison technique (JWT, Feign, Eureka)

Couplage synchrone (Feign)

MS5 → MS1 (spec de tests) · MS5 → MS3 (dépôt, verdict)

MS4 → MS2 et MS3 (données du jury)

1 base par service · aucun couplage par FK

Décomposition en microservices

Cinq services portés individuellement, plus un sixième porté par l'équipe sur une technologie avancée.

MS1 M1

Challenges

Espace entreprise, défis (spec, critères, tags, dates, cycle de vie), budget et primes, catalogue.

H2

MS2 M2

Participants

Développeurs, équipes, inscriptions et candidatures aux défis.

H2

MS3 M3

Soumissions

Dépôt GitHub, commit figé, artefacts, cycle de vie, contrôle de la date limite.

H2

MS4 M4

Évaluation

Experts, jury, grilles pondérées, scores, leaderboard, désignation des gagnants.

MySQL

MS5 ÉQUIPE

Vérification AVANCÉ

Technologie avancée : tests en bac à sable, anti-plagiat, IA d'appoint.

PostgreSQL + pgvector

MS6 M5

Réputation

Profils, réalisations vérifiées, compétences, classements.

H2

Infrastructure partagée (équipe) : **API Gateway**, **Keycloak**, **Eureka**, **Docker/Compose**, **Frontend**.

Une base par service

SERVICE	BASE	JUSTIFICATION
MS1 · Challenges	`H2`	Données relationnelles simples (défis, primes)
MS2 · Participants	`H2`	Entités légères (développeurs, équipes, candidatures)
MS3 · Soumissions	`H2`	Métadonnées de dépôts, cycle de vie
MS4 · Évaluation	`MySQL`	Scores, grilles et agrégations, donc un SGBD robuste
MS5 · Vérification	`PostgreSQL + pgvector`	Résultats de tests (JSONB) et similarité vectorielle
MS6 · Réputation	`H2`	Profils, badges, classements

> **Règle d'or :** une base par service, *aucune clé étrangère inter-services*. Les références croisées passent par identifiant et par événements.

Pas de transaction distribuée : la cohérence entre services est **à terme**, portée par les événements, et chaque service migre son propre schéma.

Communication synchrone

Frontend	→	**Gateway**

Routage et validation du JWT vers tous les services.

MS5 · Vérification	→	**MS1 · Challenges**

Récupérer la spec de tests d'acceptation du défi.

MS5 · Vérification	⇄	**MS3 · Soumissions**

Récupérer le dépôt de la soumission, puis pousser le verdict.

MS4 · Évaluation	→	**MS3 et MS2**

Données de soumission et de participants pour le jury.

> **REST via OpenFeign.** Appels directs via **Eureka**, hors Gateway. Relais du jeton pour les appels utilisateur, compte de service pour les traitements d'événement. Résilience : délais courts, réessais bornés, disjoncteur (`Resilience4j`).

Communication asynchrone

RabbitMQ via Spring Cloud Stream : le flux cœur est entièrement événementiel.

ÉVÉNEMENT	PRODUCTEUR	CONSOMMATEURS
`challenge.published`	**MS1**	MS2, MS4
`application.registered`	**MS2**	MS1, MS3
`submission.submitted`	**MS3**	MS5, MS4
`verification.completed`	**MS5**	MS3, MS4, MS2, MS6
`submission.rejected`	**MS5**	MS3, MS2
`judging.completed`	**MS4**	MS1, MS2
`winners.declared`	**MS4**	MS1, MS2, MS6
`similarity.flagged`	**MS5**	MS4, MS3

Enveloppe commune : `eventId`, `type`, `occurredAt`, `producer`, `version`, `payload`. Un groupe durable par consommateur, une file de rebut en cas d'échec, des consommateurs **idempotents** par `eventId` : livraison « au moins une fois ».

Stack technologique

LANGAGE	**Java et Spring Boot**, standard des microservices, imposé par le cours.
GATEWAY	**Spring Cloud Gateway** : routage, filtres, intégration Spring Security.
DÉCOUVERTE	**Eureka** : enregistrement, découverte, équilibrage côté client.
SÉCURITÉ	**Keycloak** : OIDC et JWT, rôles, standard éprouvé.
OBSERVABILITÉ	**Spring Boot Actuator** (`/health`), consoles Eureka et RabbitMQ, journaux corrélés par `eventId`.
COMMUNICATION	**OpenFeign et RabbitMQ** : couvre les deux modes exigés.
BASES	**H2, MySQL, PostgreSQL et pgvector** : persistance par service, pgvector pour la similarité.
CONTENEURS	**Docker et Compose** : lancement reproductible de bout en bout.
FRONTEND	**Angular 21 (Node 22)** : SPA consommant les API via la Gateway.

Répartition des tâches

M1	**MS1 · Challenges**	API défis, spec, critères, primes, catalogue, tests
M2	**MS2 · Participants**	API développeurs et équipes, inscriptions, validations
M3	**MS3 · Soumissions**	API soumissions, dépôt GitHub, cycle de vie, date limite
M4	**MS4 · Évaluation**	API jury, grilles, scores, leaderboard, gagnants
M5	**MS6 · Réputation**	API profils, badges, classements
Équipe	**MS5 + Infra + Front**	Bac à sable de tests, anti-plagiat, IA · Gateway, Keycloak, Eureka, Docker · écrans

Intégration : contrats gelés en S4, développement indépendant sur mocks, intégration S5-S9, démonstration S10.

TÂCHE M1 · MS1

MS1 · Challenges

Rôle

Gérer l'**espace entreprise** et tout le cycle de vie d'un **défi** : de sa rédaction à la désignation des gagnants.

Responsabilités

- Créer et modifier un défi : **spec**, critères, tags, dates.
- Définir les **tests d'acceptation** utilisés par le moteur de vérification.
- Gérer le **budget et les primes** (top 3).
- Publier le défi et l'exposer dans le **catalogue**.
- Accès protégé : seule l'**ENTERPRISE** publie et finance.

Données et communications

BASE	`H2` — défis, critères, primes : données relationnelles simples.
PUBLIE	`challenge.published`
ÉCOUTE	`application.registered` · `judging.completed` · `winners.declared`
FEIGN	**Appelé par MS5** pour récupérer la spec de tests du défi.

Dans le scénario : étape **01** — l'entreprise publie un défi (spec, tests, primes, date limite). Les événements reçus font avancer son cycle de vie jusqu'aux gagnants.

TÂCHE M1 · MS1

MS1 · Contrats d'API

`POST` `/api/challenges`
Créer un défi (brouillon).

`GET` `/api/challenges`
Catalogue des défis.

`GET` `/api/challenges/{id}`
Détail d'un défi.

`PUT` `/api/challenges/{id}`
Modifier un défi.

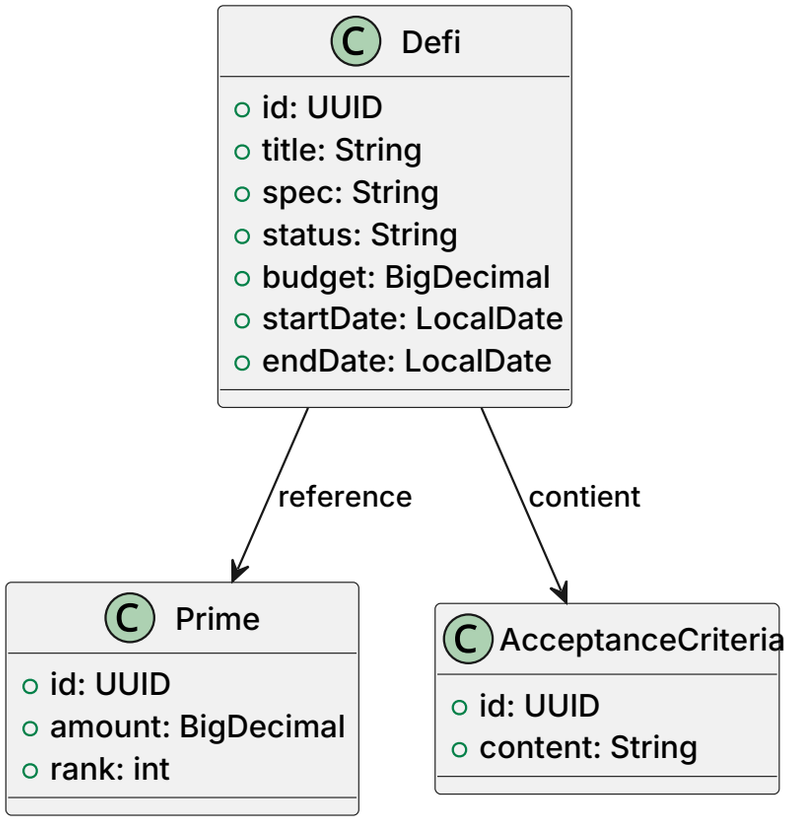
`POST` `/api/challenges/{id}/publish`
Publier → émet `challenge.published`.

`GET·PUT` `/api/challenges/{id}/acceptance-criteria`
Lire / définir les tests d'acceptation (lus par MS5).

`GET·PUT` `/api/challenges/{id}/prizes`
Lire / définir les primes du top 3.

Documentation **Swagger / OpenAPI** via `springdoc-openapi`. Tests : création, publication, validation des dates et des primes, cas d'erreur (`ErrorResponse`).

MS1 · Diagramme de classes



Défi, critères d'acceptation et primes : le modèle de données du service Challenges.

TÂCHE M2 · MS2

MS2 · Participants

Rôle

Gérer les **développeurs**, les **équipes** et leurs **inscriptions** aux défis.

Responsabilités

- Créer et consulter un **profil développeur**.
- Former une **équipe** : participation seul ou à plusieurs.
- Gérer les **candidatures** à un défi publié.
- **Valider** les inscriptions (défi ouvert, doublons, date limite).
- Suivre le statut du participant : vérifié, rejeté, gagnant.

Données et communications

BASE	`H2` — développeurs, équipes, candidatures : entités légères.
PUBLIE	`application.registered`
ÉCOUTE	`challenge.published` · `verification.completed` · `submission.rejected` · `judging.completed` · `winners.declared`
FEIGN	**Appelé par MS4** : données des participants pour le jury.

Dans le scénario : étape **02** — le développeur s'inscrit au défi, seul ou en équipe. Son inscription débloque la soumission côté MS3.

TÂCHE M2 · MS2

MS2 · Contrats d'API

`POST` `/api/developers`
Créer un profil développeur.

`GET` `/api/developers/{id}`
Consulter un développeur.

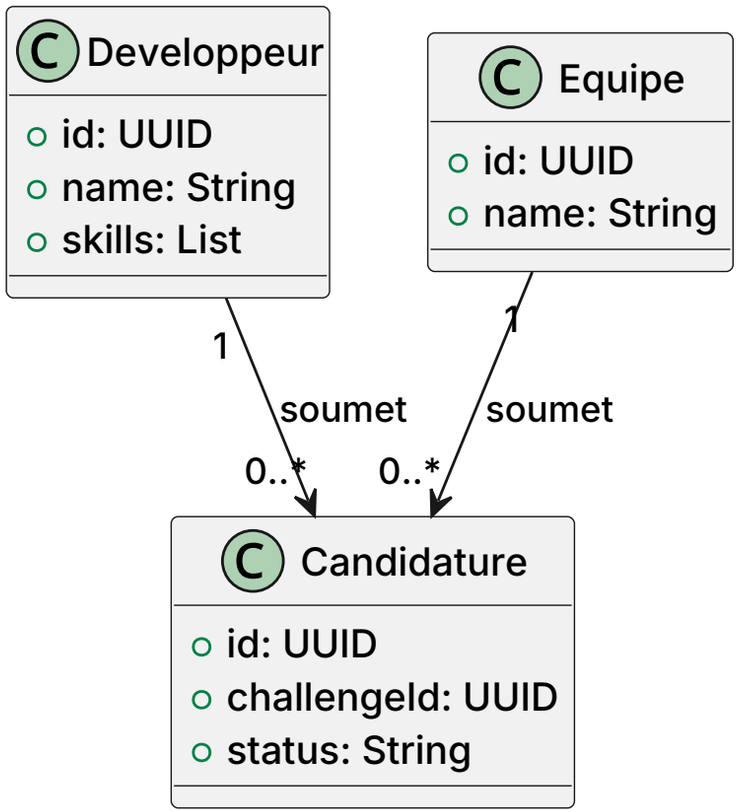
`POST` `/api/teams`
Créer une équipe.

`GET` `/api/teams/{id}`
Consulter une équipe et ses membres.

`POST·GET` `/api/challenges/{challengeId}/applications`
S'inscrire à un défi → `application.registered` ; lister les inscrits.

Documentation **Swagger / OpenAPI** via `springdoc-openapi`. Tests : inscription valide, double inscription, défi non publié, équipe incomplète.

MS2 · Diagramme de classes



Développeurs, équipes et candidatures : le modèle de données du service Participants.

TÂCHE M3 · MS3

MS3 · Soumissions

Rôle

Recevoir le **dépôt GitHub** d'un participant et suivre la soumission jusqu'au verdict.

Responsabilités

- Enregistrer le dépôt GitHub avec un **commit figé**.
- Gérer les **artefacts** de la soumission.
- Gérer le **cycle de vie** : brouillon → soumise → vérifiée ou rejetée.
- **Contrôler la date limite** : aucune soumission après échéance.
- Stocker le **verdict** renvoyé par MS5.

Données et communications

BASE	`H2` — métadonnées de dépôts, cycle de vie.
PUBLIE	`submission.submitted`
ÉCOUTE	`application.registered` · `verification.completed` · `submission.rejected` · `similarity.flagged`
FEIGN	**MS5 ⇄ MS3** : MS5 lit le dépôt puis pousse le verdict. **MS4** lit les soumissions.

> **Dans le scénario :** étape **03** — le développeur soumet son dépôt GitHub avec un commit figé. L'événement `submission.submitted` lance le flux cœur.

TÂCHE M3 · MS3

MS3 · Contrats d'API

`POST` `/api/submissions`
Créer une soumission (dépôt + commit).

`GET` `/api/submissions`
Lister les soumissions.

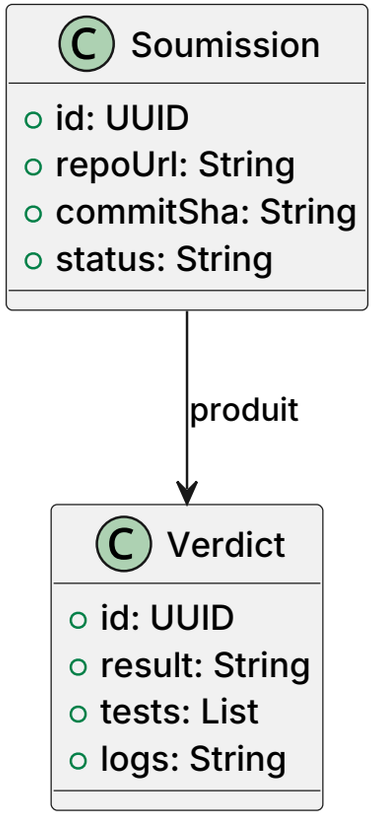
`GET` `/api/submissions/{id}`
Détail d'une soumission.

`POST` `/api/submissions/{id}/submit`
Soumettre → émet `submission.submitted`.

`GET·PUT` `/api/submissions/{id}/verdict`
Lire le verdict ; MS5 le pousse (PASS / FAIL).

Documentation **Swagger / OpenAPI** via `springdoc-openapi`. Tests : soumission après date limite, participant non inscrit, URL GitHub invalide.

MS3 · Diagramme de classes



Soumission, dépôt et verdict : le modèle de données du service Soumissions.

TÂCHE M4 · MS4

MS4 · Évaluation

Rôle

Organiser le **jury** : les experts notent les projets retenus et le service désigne les **gagnants**.

Responsabilités

- Gérer les **experts** et composer le **jury** d'un défi.
- Définir des **grilles pondérées** (critères clairs).
- Enregistrer les **scores** ; seuls les projets PASS sont notés.
- Calculer le **leaderboard**.
- **Désigner les gagnants** (top 3). Rôle : **EXPERT**.

Données et communications

BASE	`MySQL` — scores, grilles et agrégations : SGBD robuste.
PUBLIE	`judging.completed` · `winners.declared`
ÉCOUTE	`challenge.published` · `submission.submitted` · `verification.completed` · `similarity.flagged`
FEIGN	**Appelle MS3 et MS2** : données de soumission et de participants pour le jury.

Dans le scénario : étapes **05 et 06** — les experts notent les projets retenus, puis les gagnants sont désignés et récompensés. Une alerte de plagiat est visible avant la notation.

TÂCHE M4 · MS4

MS4 · Contrats d'API

`POST` `/api/experts`
Enregistrer un expert.

`POST·GET` `/api/challenges/{challengeId}/jury`
Composer / consulter le jury d'un défi.

`POST` `/api/rubrics`
Créer une grille pondérée.

`GET` `/api/rubrics`
Lister les grilles.

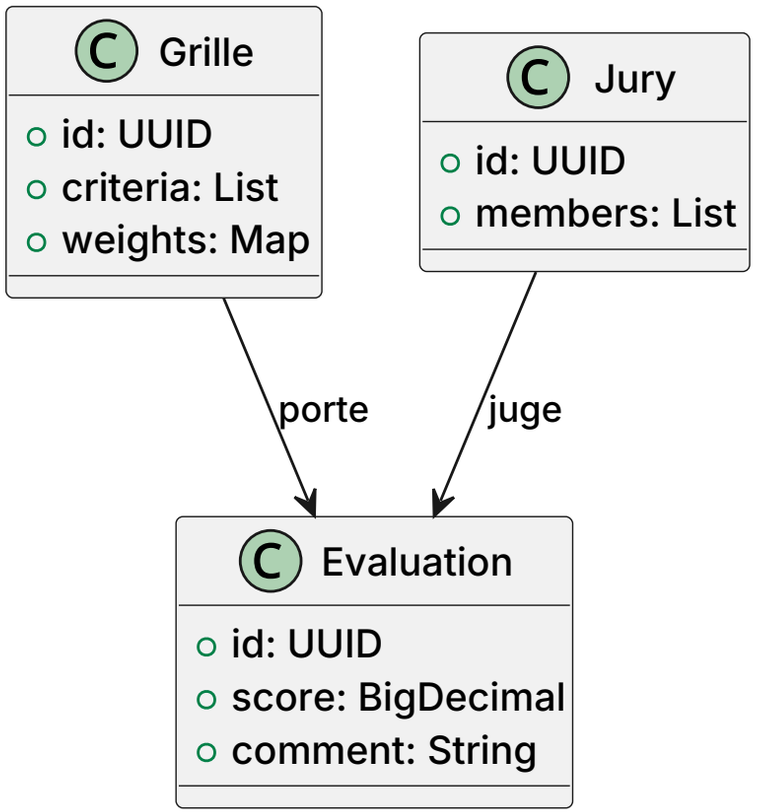
`POST` `/api/evaluations`
Noter une soumission selon une grille.

`GET` `/api/challenges/{challengeId}/leaderboard`
Classement du défi.

`POST` `/api/challenges/{challengeId}/declare-winners`
Désigner le top 3 → `winners.declared`.

Documentation **Swagger / OpenAPI** via `springdoc-openapi`. Tests : notation d'un projet FAIL refusée, poids de grille invalides, égalités au classement.

MS4 · Diagramme de classes



Grilles, jury et évaluations : le modèle de données du service Évaluation.

TÂCHE M5 · MS6

MS6 · Réputation

Rôle

Construire un **portfolio vérifiable** pour chaque développeur, à partir des résultats réels.

Responsabilités

- Gérer les **profils** de réputation.
- Enregistrer les **réalisations vérifiées** (tests PASS, victoires).
- Suivre les **compétences** et attribuer des **badges**.
- Calculer les **classements** globaux.
- Service **consommateur pur** : il ne fait qu'écouter les événements.

Données et communications

BASE	`H2` : profils, réalisations, badges.
PUBLIE	Aucun (consommateur uniquement).
ÉCOUTE	`verification.completed` · `winners.declared`
FEIGN	Aucun appel synchrone : couplage uniquement par événements.
PANNE	MS6 arrêté : les événements attendent dans RabbitMQ (file durable) et sont traités au redémarrage. Aucun service bloqué.

Dans le scénario : après les étapes **04 et 06**, chaque verdict et chaque victoire alimentent la réputation du développeur. C'est la **valeur ajoutée** pour l'embauche.

MS6 · Données et contrats d'API

Modèle de données H2

`ReputationProfile`

developerId · score · level · updatedAt

`Achievement`

id · developerId · type (TEST_PASSED | WIN) · challengeId · points · eventId (unique) · occurredAt

`Badge`

id · developerId · code (FIRST_PASS, TOP3...) · awardedAt

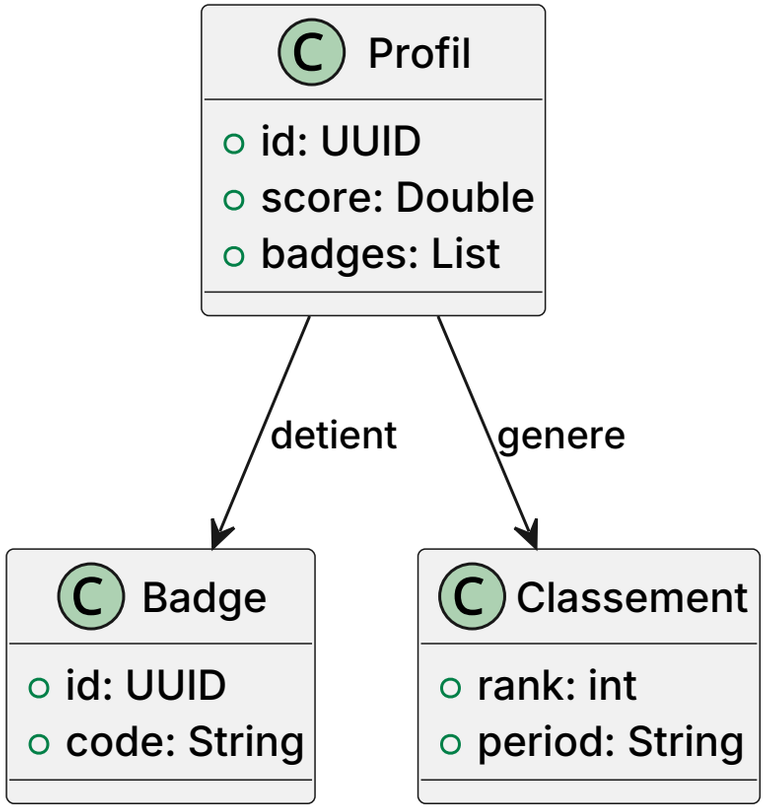
Classement : calculé à la volée (tri des profils par score), non stocké.

API REST

`GET` `/api/reputation/{developerId}`	Score et réalisations d'un développeur.	`200` · `404`
`GET` `/api/reputation/{developerId}/badges`	Badges obtenus.	`200` · `404`
`GET` `/api/reputation/rankings?page=&size=`	Classement global, paginé.	`200`
`POST` `/api/reputation/events`	Rejeu technique d'un événement. **Rôle ADMIN**.	`202` · `403` · `409`

Swagger / OpenAPI via `springdoc-openapi`. **Idempotence** : `eventId` unique, un événement reçu deux fois ne compte qu'une fois (409). **Aucune clé étrangère** : `developerId` référence MS2 par identifiant.

MS6 · Diagramme de classes



Profil, badges et classement : le modèle de données du service Réputation.

MS5 : la technologie avancée

Moteur de tests

Déclenché par `submission.submitted`, il récupère le dépôt et exécute les tests dans un **conteneur isolé**.

Verdict : `{PASS|FAIL, tests, couverture, logs}`

Isolation : `--network none`, mémoire et CPU plafonnés, utilisateur non-root, système de fichiers en lecture seule, délai maximal strict.

Anti-plagiat

Embeddings des soumissions et similarité cosinus via **pgvector** : alerte le jury avant la notation.

IA d'appoint

Résumé de soumission pour le jury, rédaction de tests à partir des exigences, appariement experts et défis.

L'IA est une amélioration, pas le cœur. `docker compose stop ms5-verification` : la plateforme reste utilisable en vérification manuelle ; seuls les badges IA et l'anti-plagiat disparaissent.

TÂCHE ÉQUIPE · MS5 VÉRIFICATION

MS5 · Données, flux et contrats

Données et communications

BASE	`PostgreSQL + pgvector` — résultats de tests en JSONB, embeddings pour la similarité.
ÉCOUTE	`submission.submitted`
PUBLIE	`verification.completed` · `submission.rejected` · `similarity.flagged`
FEIGN	→ **MS1** : spec de tests. ⇄ **MS3** : dépôt, puis verdict.

Contrats d'API

`POST` `/api/verifications`
Lancer une vérification.

`GET` `/api/verifications/{id}`
Résultat : PASS / FAIL, tests, couverture.

`GET` `/api/verifications/{id}/logs`
Logs consultables par l'équipe.

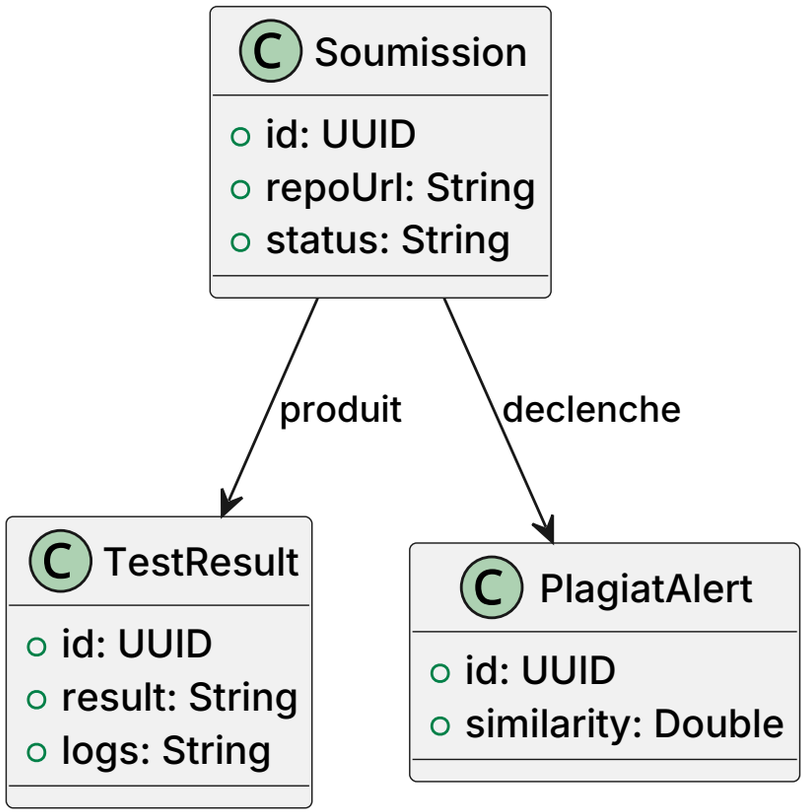
`POST` `/api/similarity/analyze`
Analyse anti-plagiat.

`GET` `/api/similarity/{submissionId}`
Soumissions similaires.

`POST` `/api/ai/summarize/{submissionId}`
Résumé IA pour le jury.

Dans le scénario : étape **04** — le moteur exécute les tests et rend un verdict. Un `FAIL` déclenche le **rejet automatique** (`submission.rejected`).

MS5 · Diagramme de classes



Vérification, résultats de tests et similarité : le modèle de données du service Vérification.

Gateway, sécurité et configuration

API Gateway

RÔLE	Point d'entrée unique, routage vers les services via Eureka.
SÉCURITÉ	Validation des JWT Keycloak, CORS, rate limiting léger.
ROUTAGE	Par chemin, ex. `/api/reputation/**` → MS6.

Keycloak

REALM	`arena` et rôles DEVELOPER, ENTERPRISE, EXPERT, ADMIN.
CLIENTS	Frontend public (Authorization Code + PKCE), gateway confidentiel.

L'authentification relève de Keycloak (infrastructure), pas d'un microservice. On le configure, on ne le code pas : chaque service valide simplement le JWT (`SecurityFilterChain`, `@PreAuthorize`). Un service d'auth maison ne serait qu'un habillage, donc un anti-pattern.

Configuration et conventions. Un `Dockerfile` par service et un `docker-compose` de bout en bout, profils `dev` et `docker`, secrets par variables d'environnement. Erreurs `ErrorResponse`, pagination `page/size/sort`, idempotence par `eventId`.

TÂCHE ÉQUIPE · INFRA + FRONT

Infrastructure et Frontend

Infrastructure

EUREKA	Enregistrement et découverte des services, équilibrage côté client.
CONFIG	**Spring Cloud Config Server** : configuration centralisée, un fichier par service.
RABBITMQ	Broker des 8 événements via Spring Cloud Stream ; file de rebut en cas d'échec.
DOCKER	Un `Dockerfile` par service, un `docker-compose` de bout en bout, profils `dev` et `docker`.
SUIVI	Spring Boot Actuator (`/health`), consoles Eureka et RabbitMQ.

Frontend · Angular 21

SPA qui passe **uniquement par la Gateway** ; connexion Keycloak (Authorization Code + PKCE).

- **Entreprise** : publier un défi, tests, primes, tableau de bord.
- **Développeur** : catalogue, inscription, soumission, verdict et logs.
- **Expert** : grilles, notation, leaderboard.
- **Tous** : profils, badges et classements.

Ordre de livraison : S5 Gateway, Eureka, Config Server et Compose local · S8 Keycloak · S9 dockerisation complète et bac à sable · S10 démonstration.

Gestion de projet et organisation Git

GitHub Projects

Backlog	À faire	En cours	Revue	Fait

- Un **label par microservice**.
- Une **milestone par séance**, de S4 à S10.
- Epics *Jalon 1* et *Jalon 2*.

Git

```
main ← develop ← feature/msX-…
```

- PR avec revue avant merge.
- Commits conventionnels : `feat(ms1): …`.
- Commits réguliers et **traçables par membre**, pour l'évaluation individuelle.
- Intégration continue (optionnelle) : build et tests à chaque PR.

Feuille de route et points de vigilance

FEUILLE DE ROUTE

S4	Validation de conception (ce document)
S5	Gateway, Eureka et Compose local
S6	Communication synchrone (Feign)
S7	Asynchrone (RabbitMQ) et pipeline de vérification
S8	Sécurité Keycloak (realm, rôles, JWT)
S9	Dockerisation complète et bac à sable
S10	Démonstration et soutenance

RISQUES ET MITIGATION

ÉLEVÉ	Exécution de code non fiable : conteneurs isolés (réseau coupé, mémoire et CPU limités, non-root, timeout). Variante : image Docker et commande fournies.
MOYEN	Périmètre large : priorisation MoSCoW, le catalogue d'événements borne les responsabilités.
MOYEN	Intégration tardive : contrats gelés en S4 et mocks WireMock.
MOYEN	Démo dépendante d'API externes : jeux de données pré-remplis, profils mock.

Merci

Entrez en arène.

PROCHAINES ÉTAPES · SQUELETTES SPRING BOOT, DOCKER-COMPOSE ET MAQUETTES FRONTEND

LIVRABLES S4 · CE DOSSIER, LE DIAGRAMME D'ARCHITECTURE ET LES CONTRATS D'API

PLACE À LA DISCUSSION.